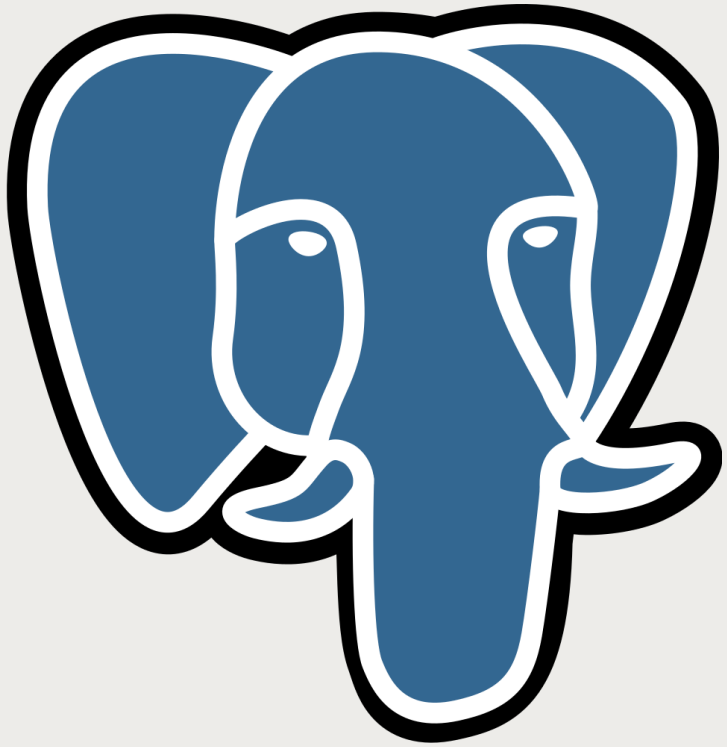
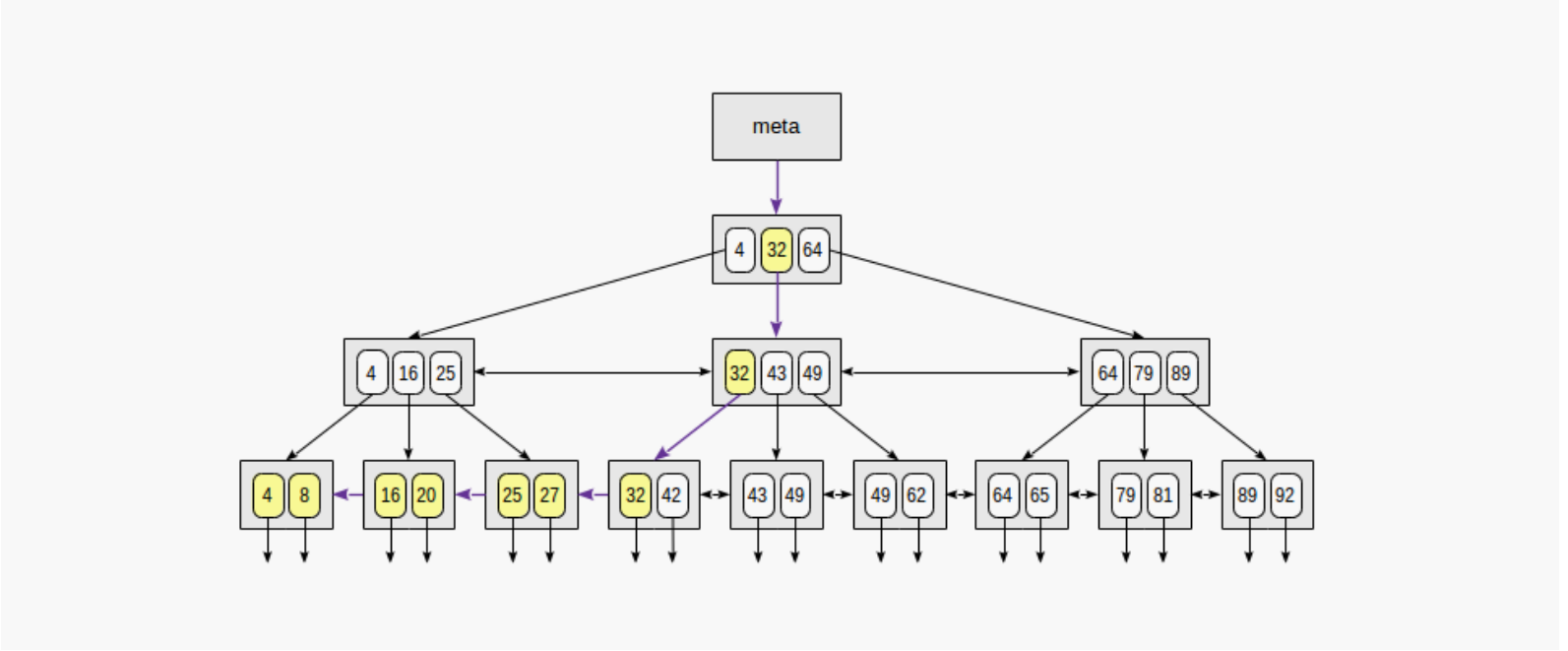
SEARCH IN POSTGRESQL



WHAT OPTIONS DO POSTGRESQL HAVE WHEN IT COMES TO SEARCH AND INDEXES?

BY FABIAN MIHLE HANSEN

POSTGRESQL DEFAULT INDEX IS B-TREE



- The «default» index type is different in different databases, PostgreSQL uses B-Tree.
- In some databases, the default index is case insensitive, but in Postgres its case sensitive.
- Works very vell for exact and prefix (on field, not tokens) search, but not for other types of search

---

SOURCES:
HTTPS://POSTGRESPRO.COM/BLOG/PGSQL/4161516
HTTPS://NEON.COM/POSTGRESQL/INDEXES

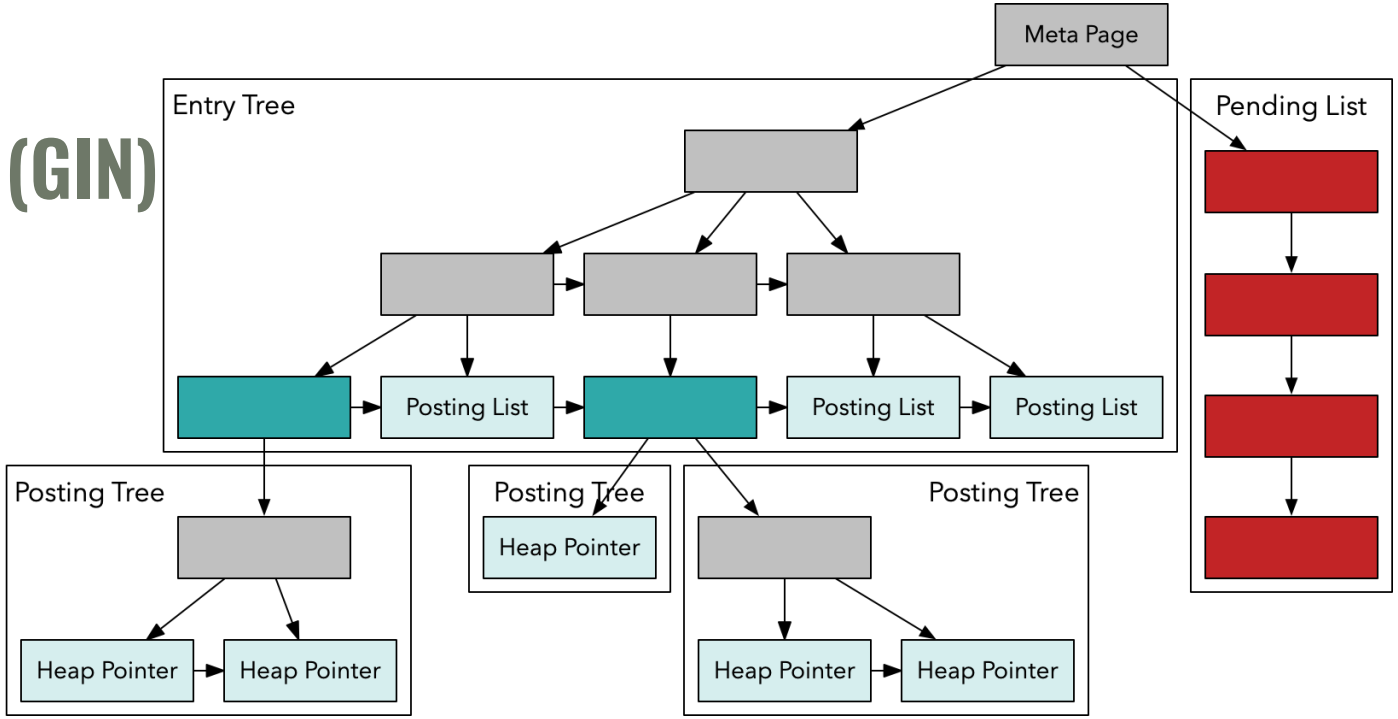
CASE SENSITIVITY

Some of the ways you can do case insensitive prefix or «exact» search:

- Specify the column as “citext” instead of “text”
    - Requires a workaround to work with for example trigram indexes.
- Create index with lower() and use lower() at query time.
    - Works on prefix/exact search.
- Use ILIKE
    - Not standard SQL
    - For slower for prefix/exact search, it does not use normal B-tree indexes.
- GIN index with trigrams index er case insensitive (More info later)
- GIN index with full text search is case insensitive (More info later)

GENERALIZED INVERTED INDEX (GIN)

- For complex/composite data fields.
  - Longer texts, JSON, Array data types etc.
- (Key, Posting List) pairs.
  - When the list of keys, or posting list get too large, it's converted to a B-tree like structure.
    Entry tree for keys, posting tree for posting list



- Requires an "Operator class" that specify what to do with a certain datatype.
  - How to tokenize, what operators are allowed (equality, greater than, similarity) etc.
  - A separate Operator for trigrams, full text search(tsvector), JSONB, arrays search etc.
- The Operator class decides if the index have for example positional data.
- Don't care about the order of the columns in the index. (B tree and some of the other do).

Note:
There is also GiST index, that can do similar things and is tree based. Its speeds up queries less, faster to create, takes up less space. It is more beneficial with overlapping data, geometry, spatial etc data. Or write heavy databases.

SOURCES:
HTTPS://WWW.POSTGRESQL.ORG/DOCS/CURRENT/GIN.HTML
HTTPS://PGANALYZE.COM/BLOG/GIN-INDEX

TRIGRAM GIN INDEX AND SIMILARITY SEARCH

- N-gram with 3 characters.
- GIN Index with “pg_trgm” operator class.
- Adds space to trigrams for better word boundary matching:
  - Cat -> “ c”, “ ca”, “cat”, “at “
- Allows for similarity() that does Jaccard similarity:
  - $\frac{|A \cap B|}{|A \cup B|}$
- Speeds up wildcard searches:
  - Example: LIKE ' %gram% ' :
    Postgres will use the trigram index to filter down rows with the trigrams, then look closer on those rows with scanning.
  - Speeds up some RegEx searches the same way.

SQL

```sql
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX articles_title_description_trgm_idx
ON articles
USING gin ((title || ' ' || description) gin_trgm_ops);
```

' %gram% ‘ --> rows containing “gra” AND “ram”

SOURCES:
HTTPS://WWW.POSTGRESQL.ORG/DOCS/CURRENT/PGTRGM.HTML
HTTPS://NEON.COM/DOCS/EXTENSIONS/PG_TRGM

FULL TEXT SEARCH

- Full text search is search that finds relevant documents using indexed lexemes and support operators for AND (&), OR (|), etc.
- Uses ‘tsvector’ (text search vector) data type, either as a normal database column(common) or directly in the index.
- Commonly used with GIN index for efficient search.
- A ‘tsvector’ stores lexems and word positions.
- Text is normalized: converted to lexemes (running -> run) and some stop words are removed.
- Uses language-specific rules or dictionary.
- Can optionally set weight to different fields in the generation of the ‘tsvector’.

“Running cats” matches “A story about a cat that runs quickly”.

SQL

```sql
CREATE TABLE blog_posts (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT NOT NULL,

    search_vector tsvector GENERATED ALWAYS AS (
        setweight(to_tsvector('english', title), 'A') ||
        setweight(to_tsvector('english', content), 'D')
    ) STORED
);

CREATE INDEX blog_posts_search_idx
ON blog_posts
USING GIN (search_vector);
```

title: Running with Cats
body: A cat runs quickly

→

tsvector:
'cat':3A,5D 'quick':7D 'run':1A,6D

SOURCES:
HTTPS://WWW.POSTGRESQL.ORG/DOCS/CURRENT/TEXTSEARCH-CONTROLS.HTML
HTTPS://NEON.COM/POSTGRESQL/INDEXES/FULL-TEXT-SEARCH

RANKED SEARCH WITH TSVECTOR

- There us is multiple alternatives for creating the query:
  - to_tsquery(): tsquery syntax, uses operators like |, &, <-> etc
  - websearch_to_tsquery(): web search like syntax, uses operators like “”, ‘AND’, ‘OR’, -word etc
  - plainto_tsquery(): treats query as AND between every word.
  - phraseto_tsquery(): search for terms in the specific given order.
- To get ranked search, use ‘ts_rank()’ or ‘ts_rank_cd()’

SQL

```sql
SELECT id, title, content,
  ts_rank_cd(
    search_vector,
    websearch_to_tsquery('english', 'running cats OR dogs'),
    1) AS rank
FROM blog_posts
WHERE search_vector @@
  websearch_to_tsquery('english','running cats OR dogs')
ORDER BY rank DESC;
```

- ‘ts_rank_cd()’ uses relative positions between terms while ‘ts_rank()’ does not.
  - Ranking can be normalized by, for example:
    - rank / (1+log(document length))
    - rank / harmonic distance between extents
    - rank / (1 + log(the number of unique words in document))

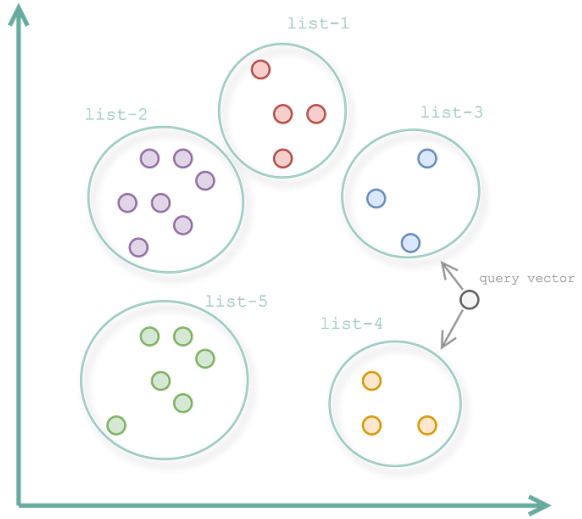
SOURCES:
HTTPS://WWW.POSTGRESQL.ORG/DOCS/CURRENT/TEXTSEARCH-CONTROLS.HTML
HTTPS://NEON.COM/POSTGRESQL/INDEXES/FULL-TEXT-SEARCH

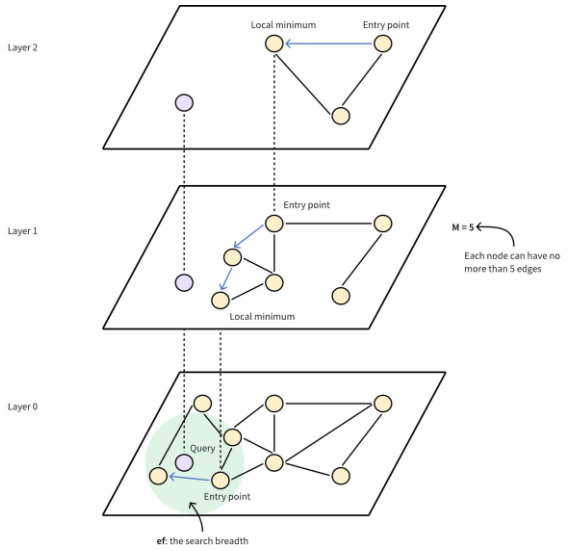
VECTOR SEARCH

- There is also an open-source extension that is “pgvector” (postgres vector)
- ‘pgvector’ is numerical vector embeddings of the text, like in the lectures.
  - 'Cats are popular pets’ → '[0.9, 0.8, 0.7….]'
- Can be used with cosine similarity, Manhatten, machine learning etc and more.
- Can be used with indexes like HNSW, ICFFLAT, where both are approximate nearest neighbour.
  - HNSW is a multiplayer graph index
  - ICFFlat is a centroid based index

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  embedding VECTOR(1536) NOT NULL
);
```





SOURCES:
HTTPS://WWW.DATABRICKS.COM/BLOG/WHAT-IS-PGVECTOR
HTTPS://GITHUB.COM/PGVECTOR/PGVECTOR

THE END

I HOPE YOU LEARNED SOMETHING USEFUL